Projektrapport

StudyFlow / Projektuppgift

DT071G, Programmering i C#.NET

Författare: Fredrika Stjernlöf, frst2501@student.miun.se
Termin, år: HT, 2026



Mittuniversitetet

MID SWEDEN UNIVERSITY

1 Sammanfattning

Detta projekt har genomförts inom kursen Programmering i C#.NET vid Mittuniversitetet. Syftet var att tillämpa och utveckla de kunskaper som förvärvats under kursen genom att skapa en webbaserad applikation. Projektet inriktades på att utveckla en studieplanerare som hjälper studenter att organisera sina kurser, uppgifter och checklistor samt prioritera sitt studiearbete. Resultatet fick namnet StudyFlow.

Applikationen har utvecklats med C# och ASP.NET Core Razor Pages, med Entity Framework Core och SQLite för databashantering. Rapporten beskriver projektets planering, design och utvecklingsprocess samt hur de olika funktionerna har implementerats. Vidare behandlas de tekniska lösningar som valts och de utmaningar som uppstått under arbetets gång.

Resultatet är en fungerande webbapplikation med personlig användarhantering, kurs- och uppgiftshantering, checklistor och en regelbaserad prioriteringsalgoritm. Projektet har även gett praktisk erfarenhet av att utveckla en sammanhängande applikation där databas, serverlogik och användargränssnitt samverkar.

2 Innehållsförteckning

1 Sammanfattning 2
2 Innehållsförteckning 3
3 Introduktion 4
3.1 Bakgrund och problemmotivering 4
3.2 Detaljerad problemformulering 4
3.3 Syfte och mål 4
3.4 Avgränsningar 5
4 Teori 6
4.1 C# och .NET 6
4.2 ASP.NET Core och Razor Pages 6
4.3 Entity Framework Core och SQLite 6
4.4 ASP.NET Core Identity 6
4.5 Regelbaserad prioritering 7
5 Metod 8
5.1 Planering och förarbete 8
5.2 Design och prototyp 8
5.3 Utvecklingsmiljö och verktyg 8
5.4 Arbetsprocess 8
5.5 Testning och kvalitetssäkring 9
6 Konstruktion 10
6.1 Projektstruktur och arkitektur 10
6.2 Databas och datamodeller 10
6.3 Användarhantering och autentisering 11
6.4 Kurshantering 12
6.5 Uppgiftshantering och checklistor 13
6.6 Prioriteringsalgoritmen 13
6.7 Dashboard och navigation 14
6.8 Gränssnitt och responsiv design 15
7 Resultat 16
8 Slutsatser 17
9 Källförteckning 18
Bilaga A 19

3 Introduktion

3.1 Bakgrund och problemmotivering

Att studera innebär ofta att man behöver hantera flera kurser och uppgifter parallellt, med olika deadlines och varierande arbetsbelastning. Det kan vara svårt att få en tydlig överblick över vad som behöver göras och vilka uppgifter som bör prioriteras, särskilt när flera moment pågår samtidigt.

Det finns redan flera digitala verktyg för att organisera arbete och studier. Under en tidigare kurs i projektledning använde vi bland annat Microsoft Teams för att planera och följa upp projektarbete med hjälp av Kanban-tavlor. Det arbetet väckte mitt intresse för hur digitala verktyg kan användas för att skapa struktur och överblick, men också tanken på att utforma något eget.

Jag har ett stort intresse för design och användarupplevelse och ville skapa en studieplanerare med ett mer personligt uttryck. I stället för ett traditionellt gränssnitt med tabeller och listor ville jag arbeta med en digital studiealmanacka, där anteckningsböcker, anslagstavlor och Post-it-lappar får representera olika delar av studierna. Tanken var att kombinera praktiska funktioner med ett gränssnitt som känns både användbart och inspirerande.

Idén utvecklades till StudyFlow, en personlig studieplanerare där användaren kan samla sina kurser, planera uppgifter och följa sitt arbete. En central del av projektet var också att utveckla en prioriteringsalgoritm som tar hänsyn till flera faktorer och hjälper användaren att avgöra vilka uppgifter som bör få mest uppmärksamhet.

3.2 Detaljerad problemformulering

Inför uppstarten av projektet fanns flera problem som skulle lösas:

- Skapa funktionalitet för att hantera kurser och studieuppgifter
- Möjliggöra planering med deadlines, prioritet och uppskattad arbetstid
- Skapa checklistor för att dela upp större uppgifter i mindre delmoment
- Utveckla en algoritm som beräknar och rekommenderar vilka uppgifter som bör prioriteras
- Skapa en dashboard som ger användaren en samlad översikt
- Implementera registrering, inloggning och användarspecifik datahantering
- Lagra kurser, uppgifter och checklistor i en databas
- Skapa ett responsivt och lättnavigerat användargränssnitt

Utöver de funktionella och tekniska kraven fanns ett designmål om att ge StudyFlow en egen visuell identitet, inspirerad av en fysisk studiealmanacka. Gränssnittet skulle kombinera tydlig struktur med ett mer personligt uttryck.

3.3 Syfte och mål

Syftet med projektet var att tillämpa och vidareutveckla de kunskaper jag fått under kursen Programmering i C#.NET genom att utveckla en större, sammanhängande applikation. Jag ville framför allt få en bättre förståelse för hur C# kan användas i webbutveckling och hur olika delar av en applikation samverkar.

Målet var att utveckla en fungerande, webbaserad studieplanerare där användaren kan skapa och hantera kurser, uppgifter och checklistor genom ett personligt konto. Applikationen skulle också innehålla en egenutvecklad, regelbaserad prioriteringsalgoritm som rekommenderar vilka uppgifter användaren bör fokusera på utifrån deadline, prioritet, status och uppskattad arbetstid.

Utöver de tekniska funktionerna ville jag lägga stor vikt vid gränssnittets utformning och skapa en applikation där funktionalitet och visuell design samverkar.

3.4 Avgränsningar

Projektet avgränsades till att utveckla en personlig studieplanerare för enskilda användare. Funktioner för samarbete mellan användare, delning av uppgifter och integration med externa kalendrar ingick inte i projektet.

Fokus låg på att skapa en fungerande applikation med en så komplett användarupplevelse som möjligt, inklusive registrering, inloggning och användarspecifik datahantering. Däremot konfigurerades ingen extern e-posttjänst för exempelvis kontobekräftelser, och publicering på en extern webbserver ingick inte i projektets omfattning.

4 Teori

4.1 C# och .NET

C# är ett objektorienterat och starkt typat programmeringsspråk som utvecklats av Microsoft. Språket används för att utveckla flera typer av applikationer, bland annat webbapplikationer, skrivbordsprogram och mobilapplikationer. C# har stöd för exempelvis klasser, objekt, arv och gränssnitt, vilket gör det möjligt att strukturera och återanvända kod [1] [2].

.NET är en utvecklingsplattform med öppen källkod som används för att bygga och köra applikationer. Plattformen är oberoende av operativsystem och kan användas på bland annat Windows, macOS och Linux. Den innehåller en körmiljö, bibliotek och utvecklingsverktyg som tillsammans ger stöd för utveckling i C# och andra programmeringsspråk [2].

C# och .NET används ofta tillsammans, där C# utgör programmeringsspråket och .NET tillhandahåller den miljö och de verktyg som behövs för att utveckla och köra applikationen. För webbutveckling finns även ramverket ASP.NET Core, som bygger på .NET-plattformen [2].

4.2 ASP.NET Core och Razor Pages

ASP.NET Core är ett webbramverk inom .NET som används för att utveckla webbapplikationer och webbtjänster med C#. Ramverket har stöd för bland annat routning, hantering av HTTP-förfrågningar och autentisering [3].

Razor Pages är en del av ASP.NET Core och bygger på ett sidbaserat arbetssätt där varje sida vanligtvis består av en Razor-fil (.cshtml) och en tillhörande C#-klass (.cshtml.cs). Razor-filen används för att kombinera HTML med dynamiskt innehåll, medan C#-klassen hanterar sidans logik och inkommande förfrågningar. Detta gör det möjligt att utveckla användargränssnitt och serverlogik inom samma projekt [4].

4.3 Entity Framework Core och SQLite

Entity Framework Core (EF Core) är ett ramverk för objektrelationell mappning (ORM) som gör det möjligt att arbeta med databaser genom C#-objekt i stället för att skriva SQL-frågor manuellt. Med hjälp av datamodeller och en databaskontext kan applikationen skapa, läsa, uppdatera och ta bort information i databasen [5].

SQLite är en relationsdatabas som lagrar information i en lokal fil och inte kräver någon separat databasserver. Databasen kan användas tillsammans med EF Core genom en särskild databasprovider. EF Core har även stöd för migreringar, vilket gör det möjligt att skapa och uppdatera databasens struktur när applikationens datamodeller förändras [5] [6].

4.4 ASP.NET Core Identity

ASP.NET Core Identity är ett ramverk för hantering av användarkonton och autentisering i ASP.NET Core-applikationer. Det innehåller färdiga funktioner för bland annat registrering, inloggning, lösenordshantering och hantering av användaruppgifter.

Identity kan integreras med Entity Framework Core för att lagra användarkonton och tillhörande information i en databas. Ramverket har även stöd för auktorisering, vilket gör

det möjligt att begränsa åtkomsten till vissa sidor och funktioner beroende på om användaren är inloggad eller har rätt behörighet [7].

4.5 Regelbaserad prioritering

En poängmodell (Scoring Model) är en metod för att jämföra och rangordna olika alternativ utifrån flera kriterier. Genom att tilldela kriterierna poäng och väga samman dessa kan ett totalt värde beräknas för varje alternativ. Modellen kan anpassas efter vilka faktorer som är relevanta för den aktuella bedömningen [8].

En regelbaserad variant kan använda fördefinierade villkor för att avgöra hur många poäng varje kriterium ska bidra med. Poängen summeras sedan till ett sammanlagt värde som används för att rangordna alternativen.

En fördel med poängbaserad prioritering är att det går att följa hur olika kriterier påverkar resultatet. Samtidigt är rangordningen beroende av vilka kriterier och poängvärden som har valts.

5 Metod

5.1 Planering och förarbete

Arbetet inleddes med att ta fram en projektplan där applikationens syfte, funktioner, tekniska lösningar och avgränsningar definierades. Målet var att skapa en tydlig grund för utvecklingen och identifiera vilka delar som behövde implementeras för att StudyFlow skulle fungera som en sammanhängande applikation.

Projektet delades upp i mindre arbetsmoment som omfattade planering och design, uppbyggnad av projektets grundstruktur, databashantering, användarhantering och utveckling av applikationens funktioner. Prioriteringsalgoritmen och det visuella gränssnittet utgjorde också viktiga delar av arbetet.

Projektplanen användes som utgångspunkt under utvecklingen, men arbetet anpassades efter de behov och tekniska utmaningar som uppstod längs vägen.

5.2 Design och prototyp

För att planera applikationens gränssnitt användes Figma. Arbetet inleddes med att skapa en moodboard för att samla visuella referenser och definiera en övergripande designriktning. Därefter togs wireframes fram för att planera sidornas struktur, placering av funktioner och navigation mellan olika vyer (se bilaga A).

Utifrån dessa skapades mockups för att visualisera hur det färdiga gränssnittet skulle kunna se ut. Dessa användes som stöd under utvecklingen och gav en tydligare bild av hur applikationens olika delar skulle utformas och samverka (se bilaga A).

Designarbetet fortsatte även under implementationen, där gränssnittet justerades och vidareutvecklades i takt med att funktionerna byggdes och testades.

5.3 Utvecklingsmiljö och verktyg

StudyFlow utvecklades i Visual Studio Code med C# och .NET 10. Applikationen byggdes med ASP.NET Core Razor Pages, vilket gjorde det möjligt att utveckla både användargränssnitt och serverlogik inom samma projekt. HTML, SCSS och JavaScript användes för gränssnittets struktur, utseende och interaktiva funktioner. Bootstrap användes som stöd för layout och responsiv design.

För datalagring användes SQLite tillsammans med Entity Framework Core. Datamodeller och databasmigreringar hanterades genom EF Core, medan ASP.NET Core Identity användes för registrering, inloggning och hantering av användarkonton.

Under utvecklingen användes .NET CLI i terminalen för att bland annat skapa och köra projektet, hantera paket och uppdatera databasen. Git och GitHub användes för versionshantering och lagring av projektets källkod.

5.4 Arbetsprocess

Projektet utvecklades iterativt, där funktioner implementerades, testades och vidareutvecklades successivt. Arbetet inleddes med att skapa projektets grundstruktur och konfigurera databasen och användarhanteringen. Därefter utvecklades funktionerna för att hantera kurser, uppgifter och checklistor.

För att hålla koden organiserad delades funktionaliteten upp i datamodeller, services och Razor Pages. Under utvecklingens gång omstrukturerades även delar av koden genom att funktionalitet flyttades till separata filer, eftersom vissa filer hade blivit omfattande och svåra att överblicka. Datamodellerna användes för att representera informationen som lagrades i databasen, medan services ansvarade för applikationens logik och datahantering. Razor Pages användes för att hantera användarinteraktion och presentera information i gränssnittet.

När de grundläggande funktionerna fungerade utvecklades prioriteringsalgoritmen och dashboarden, där information från applikationens olika delar sammanställdes. Under arbetets gång testades funktionerna lokalt och eventuella problem åtgärdades innan utvecklingen fortsatte.

Arbetsprocessen innebar även att växla mellan programmering och gränssnittsutveckling, eftersom funktionalitet och design behövde anpassas till varandra.

5.5 Testning och kvalitetssäkring

Testning genomfördes löpande under utvecklingen genom att applikationen kördes lokalt och funktionerna kontrollerades direkt i webbläsaren. När nya funktioner implementerades testades de för att säkerställa att de fungerade som avsett och samverkade med övriga delar av systemet.

Testerna omfattade bland annat registrering och inloggning, hantering av kurser och uppgifter samt möjligheten att lägga till, uppdatera och ta bort information. Även checklistornas funktionalitet och prioriteringsalgoritmens beräkningar kontrollerades. Särskild uppmärksamhet lades vid att användare endast skulle kunna komma åt sina egna uppgifter och kurser.

Gränssnittet testades också i webbläsaren för att kontrollera navigation, formulär, validering och responsiv layout. Tillgängligheten granskades med hjälp av verktygen Silktide och WAVE för att identifiera eventuella tillgänglighetsproblem. Fel och brister som upptäcktes åtgärdades successivt som en del av den iterativa utvecklingsprocessen.

6 Konstruktion

6.1 Projektstruktur och arkitektur

StudyFlow utvecklades som en ASP.NET Core-applikation med Razor Pages och C#, där användargränssnitt och serverlogik hanteras inom samma projekt. Projektet organiserades i separata mappar för datamodeller, databashantering, tjänster, sidlogik och statiska filer (se figur 1).

I mappen *Pages* finns applikationens Razor Pages, där varje sida består av en .cshtml-fil för gränssnittet och en tillhörande .cshtml.cs-fil för sidans logik. Sidorna organiserades efter funktion, exempelvis *Courses* för kurshantering och *Tasks* för uppgiftshantering. Gemensamma gränssnittskomponenter placerades i *Pages/Shared* som partial views, bland annat för uppgiftskort, formulär och dialogrutor.

Applikationens datamodeller placerades i *Models*, medan databaskontexten *ApplicationDbContext* finns i mappen *Data*. För att separera databashantering och beräkningslogik från sidornas PageModels skapades ett tjänstelager i *Services*. Här finns bland annat *CourseService*, *TaskService*, *ChecklistService* och *TaskPriorityService*. Tjänsterna registrerades i *Program.cs* och görs tillgängliga för sidornas PageModels genom dependency injection.

JavaScript-filer placerades i *wwwroot/js*, medan SCSS-koden delades upp i separata filer för bland annat layout, komponenter och enskilda sidor.

Denna struktur växte fram under utvecklingen. Inledningsvis låg en större del av funktionaliteten direkt i Razor-filerna och deras PageModels. När framför allt kurssidans filer blev omfattande genomfördes en refaktorering där gränssnitts-komponenter, JavaScript och databashantering delades upp i mindre delar med tydligare ansvarsområden. På så sätt kunde befintlig funktionalitet behållas samtidigt som kodstrukturen blev mer överskådlig.

```
StudyFlow/
├── Data/
│   └── ApplicationDbContext.cs
├── Models/
│   ├── ChecklistItem.cs
│   ├── Course.cs
│   └── StudyTask.cs
├── Pages/
│   ├── Courses/
│   │   ├── Details.cshtml
│   │   ├── Details.cshtml.cs
│   │   ├── Index.cshtml
│   │   └── Index.cshtml.cs
│   ├── Tasks/
│   │   ├── Index.cshtml
│   │   └── Index.cshtml.cs
│   ├── Shared/
│   │   ├── _CourseCard.cshtml
│   │   ├── _CreateTaskForm.cshtml
│   │   ├── _DeleteTaskDialog.cshtml
│   │   ├── _EditTaskDialog.cshtml
│   │   ├── _Layout.cshtml
│   │   ├── _LoginPartial.cshtml
│   │   ├── _TaskCard.cshtml
│   │   ├── _TaskCardCompact.cshtml
│   │   └── _TaskDetailsDialog.cshtml
│   ├── Index.cshtml
│   └── Index.cshtml.cs
├── Services/
│   ├── ChecklistService.cs
│   ├── CourseService.cs
│   ├── TaskPriorityService.cs
│   └── TaskService.cs
├── ViewModels/
│   ├── StudyTaskInputModel.cs
│   └── TaskPriorityResult.cs
├── scss/
│   ├── abstracts/
│   ├── base/
│   ├── components/
│   ├── layout/
│   └── pages/
├── wwwroot/
│   └── js/
│       ├── course-details.js
│       ├── courses.js
│       ├── task-form-validation.js
│       └── tasks.js
└── Program.cs
```

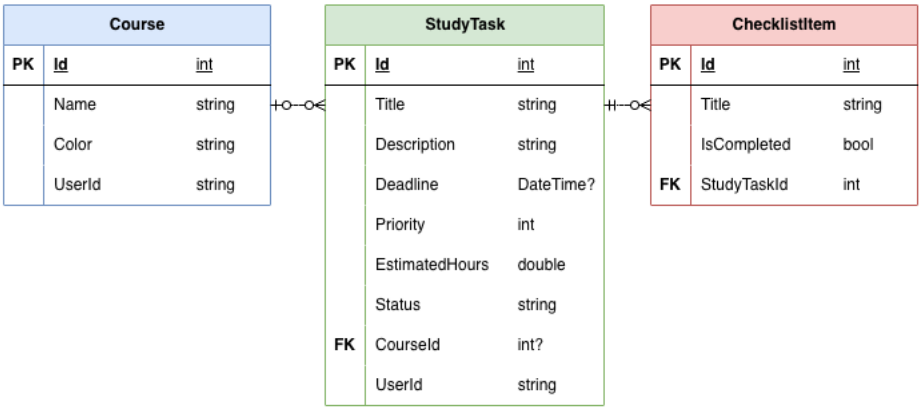
Figur 1. Förenklad översikt över StudyFlows projektstruktur med fokus på applikationens egenutvecklade filer och komponenter.

6.2 Databas och datamodeller

För att lagra användarnas kurser, uppgifter och checklistor skapades tre datamodeller i mappen *Models*: *Course.cs*, *StudyTask.cs* och *ChecklistItem.cs*. Varje modell definierar vilka egenskaper som ska lagras, exempelvis kursnamn, uppgiftens deadline, prioritet, status och checklistpunkternas färdigställande.

Relationerna mellan modellerna utformades så att en kurs kan innehålla flera uppgifter och en uppgift kan innehålla flera checklistpunkter. En uppgift kan även skapas utan att

tillhöra någon kurs, vilket möjliggjordes genom att egenskapen CourseId gjordes nullable. Varje checklistpunkt måste däremot vara kopplad till en uppgift (se figur 2).



Figur 2. Datamodeller och relationer i StudyFlows Databas.

Modellerna registrerades i *ApplicationDbContext.cs*, där Entity Framework Core används för kommunikationen med SQLite-databasen. Databasens tabeller skapades genom migreringar med kommandona dotnet ef migrations add och dotnet ef database update. När relationerna senare vidareutvecklades infördes även Cascade Delete, vilket innebär att tillhörande uppgifter och checklistpunkter automatiskt raderas när en kurs tas bort.

Varje kurs och uppgift kopplas till den inloggade användarens ID. Vid hämtning och ändring av information kontrolleras användartillhörigheten i applikationens tjänster, så att användare endast kan hantera sina egna uppgifter och kurser.

6.3 Användarhantering och autentisering

För att möjliggöra personliga användarkonton skapades StudyFlow med stöd för ASP.NET Core Identity. I *Program.cs* konfigurerades användarhanteringen med krav på kontobekräftelse vid inloggning. Databasklassen *ApplicationDbContext.cs* ärver från IdentityDbContext, vilket gör det möjligt att lagra användarkonton i samma SQLite-databas som kurser och uppgifter. Eftersom ingen extern e-posttjänst används hanteras kontobekräftelsen lokalt under utvecklingen.

För att skilja användarnas information åt innehåller modellerna *Course.cs* och *StudyTask.cs* egenskapen UserId. I sidornas PageModels, exempelvis *Pages/Tasks/Index.cshtml.cs*, hämtas den inloggade användarens ID från autentiseringsinformationen (se figur 3). Detta skickas vidare till *TaskService.cs*, som använder ID för att hämta och hantera användarens egna uppgifter. Motsvarande kontroller genomförs för kurser genom *CourseService.cs*. Sidor som kräver inloggning skyddas dessutom med attributet [Authorize].

```
public async Task OnGetAsync()
{
    if (User.Identity?.IsAuthenticated != true)
    {
        return;
    }

    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

    if (userId == null)
    {
        return;
    }
```

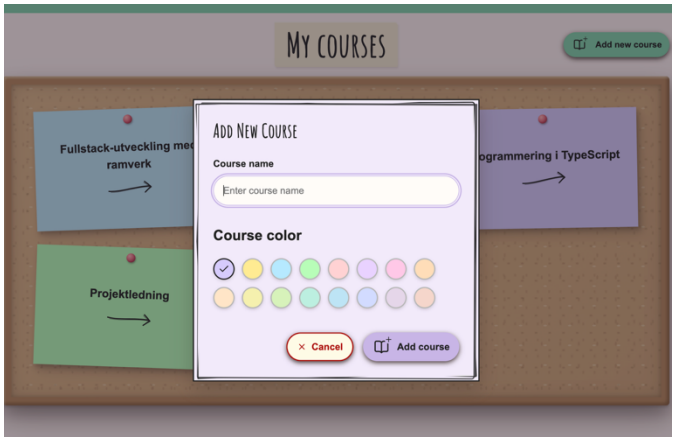
Figur 3. Kontroll av inloggningsstatus och hämtning av användar-ID i Index.cshtml.cs

Startsidan byggdes för att visa olika innehåll beroende på användarens inloggningsstatus. I Pages/Index.cshtml används villkoret @if (User.Identity?.IsAuthenticated == true) för att kontrollera om användaren är inloggad. Om villkoret är uppfyllt visas den personliga dashboarden, annars visas en välkomstsida med länkar till registrering och inloggning. För att möjliggöra detta togs attributet [Authorize] bort från startsidans PageModel, medan övriga användarspecifika sidor fortsatt kräver att användaren är inloggad.

6.4 Kurshantering

Kurshanteringen utvecklades för att användaren skulle kunna organisera sina studier efter olika kurser. Den implementerades med CRUD-operationer för att skapa, läsa, uppdatera och radera kursinformation. Funktionen byggdes med Razor Pages och består huvudsakligen av en kursöversikt i *Pages/Courses/Index.cshtml* och en detaljsida i *Pages/Courses/Details.cshtml*. Tillhörande PageModels hanterar användarens anrop och använder *CourseService.cs* för att kommunicera med databasen.

På kursöversikten kan användaren skapa en ny kurs genom ett formulär i en dialogruta. Formuläret innehåller fält för kursnamn och färgval, där färgerna hämtas från en fördefinierad lista i modellen *Course.cs*. När formuläret skickas kontrolleras inmatningen i sidans PageModel innan kursen sparas via metoden CreateCourseAsync() i *CourseService.cs*. Den nya kursen kopplas samtidigt till den inloggade användaren.



Figur 4. Kursöversikt i StudyFlow med dialogruta för att skapa en ny kurs.

Befintliga kurser hämtas genom GetUserCoursesAsync() och presenteras som kurskort med hjälp av den återanvändbara komponenten *_CourseCard.cshtml*. Genom att klicka på ett kurskort öppnas kursens detaljsida, där tillhörande uppgifter visas och användaren kan redigera eller ta bort kursen.

Redigering och borttagning hanteras genom separata formulär och metoder i detaljsidans PageModel. Dessa anropar UpdateCourseAsync() respektive DeleteCourseAsync() i *CourseService.cs*, där det även kontrolleras att kursen tillhör den inloggade användaren. Vid borttagning raderas även kursens tillhörande uppgifter och checklistpunkter genom de relationer som konfigurerats i databasen.

JavaScript används för att öppna och stänga dialogrutorna genom att hämta HTML-element med getElementById() och hantera klickhändelser med addEventListener(), medan själva formulärhanteringen och databasoperationerna utförs på serversidan.

6.5 Uppgiftshantering och checklistor

Uppgiftshanteringen utvecklades för att användaren skulle kunna skapa, visa, redigera och ta bort studieuppgifter. Funktionaliteten byggdes med Razor Pages och är tillgänglig både på den gemensamma uppgiftssidan i *Pages/Tasks/Index.cshtml* och på detaljsidan för respektive kurs, där användaren kan skapa och hantera uppgifter. Befintliga uppgifter kan även visas och redigeras direkt från den personliga dashboarden. Databasoperationerna samlades i *TaskService.cs*, medan sidornas PageModels hanterar formulär, validering och anrop till tjänsten.

För att skapa en uppgift används ett formulär där användaren kan ange titel, beskrivning, deadline, prioritet, uppskattad arbetstid och eventuell kurstillhörighet (se figur 5). Formuläret placerades i den återanvändbara komponenten *_CreateTaskForm.cshtml*, så att samma funktion kan användas från flera sidor. Inmatningen kontrolleras innan uppgiften sparas i databasen genom *TaskService.cs*.



Figur 5. Formulär för att skapa en studieuppgift med tillhörande checklistpunkter.

Varje uppgift kan även innehålla en checklista med mindre delmoment. Checklistpunkterna lagras i en separat datamodell och hanteras genom *ChecklistService.cs*. Användaren kan lägga till, markera som färdiga och ta bort checklistpunkter. För att uppdatera en checklistpunkt utan att ladda om hela sidan används JavaScript som skickar en begäran till en handler i sidans PageModel. Ändringen sparas via tjänstelagret och gränssnittet uppdateras direkt. Samma princip används för att uppdatera uppgifternas status, där JavaScript och asynkrona anrop gör det möjligt att genomföra ändringar utan att ladda om sidan.

För att göra uppgiftshanteringen mer överskådlig skapades återanvändbara partial views för uppgiftskort och dialogrutor. Bland annat används *_TaskCard.cshtml* för att presentera uppgifter och separata dialogrutor för att visa detaljer, redigera och bekräfta borttagning. JavaScript hanterar dialogrutornas interaktioner, medan uppgifternas innehåll och status sparas på serversidan.

6.6 Prioriteringsalgoritmen

För att hjälpa användaren att avgöra vilka studieuppgifter som bör prioriteras utvecklades en regelbaserad prioriteringsalgoritm i *TaskPriorityService.cs*. Algoritmen beräknar ett poängvärde för varje uppgift utifrån fyra faktorer: prioritet, deadline, status och uppskattad arbetstid.

Beräkningen genomförs i metoden CalculateScore(), där varje faktor tilldelas ett visst antal poäng (se figur 6). Uppgifter med hög prioritet får fler poäng än uppgifter med låg prioritet, medan en nära förestående eller passerad deadline höjer poängen ytterligare. Pågående uppgifter får också ett poängtillägg, liksom uppgifter med kortare uppskattad arbetstid. Poängen summeras till ett totalvärde som används för att jämföra uppgifterna.

I metoden GetTodaysFocusAsync() hämtas användarens uppgifter och färdigställda uppgifter filtreras bort. De återstående uppgifterna rangordnas efter sitt beräknade poängvärde, och uppgiften med högst poäng väljs ut som *Today's focus*. Vid lika poäng används deadline för att avgöra ordningen.

Algoritmen används på den personliga dashboarden, den gemensamma uppgiftssidan och de enskilda kurssidorna. På dashboarden och den gemensamma uppgiftssidan väljs *Today's focus* utifrån användarens samtliga uppgifter, medan prioriteringen på en kurssida endast omfattar uppgifter som tillhör den aktuella kursen. En uppgift kan därför vara högst prioriterad inom en kurs utan att vara den högst prioriterade uppgiften totalt sett. Genom att samla beräkningen i en separat tjänst kan samma prioriteringslogik återanvändas i olika delar av applikationen.

```
public class TaskPriorityService
{
    1 reference
    public int CalculateScore(StudyTask task)
    {
        int score = 0;

        // 1. Priority
        score += task.Priority switch
        {
            1 => 10, // Low priority
            2 => 20, // Medium priority
            3 => 30, // High priority
            _ => 0
        };

        // 2. Deadline
        if (task.Deadline.HasValue)
        {
            int daysLeft = (task.Deadline.Value.Date - DateTime.Today).Days;

            if (daysLeft <= 0)
                score += 40;
            else if (daysLeft <= 3)
                score += 30;
            else if (daysLeft <= 7)
                score += 20;
            else
                score += 10;
        }

        // 3. Status
        if (task.Status == "InProgress")
        {
            score += 15;
        }

        // 4. Estimated time
        if (task.EstimatedHours > 0 && task.EstimatedHours <= 2)
        {
            score += 10;
        }
        else if (task.EstimatedHours > 2 && task.EstimatedHours <= 5)
        {
            score += 5;
        }

        return score;
    }
```

Figur 6. Metoden CalculateScore() i TaskPriorityService.cs, där prioriteringspoängen beräknas utifrån fyra faktorer.

6.7 Dashboard och navigation

Den personliga dashboarden utvecklades som applikationens huvudsakliga startsida för inloggade användare. Gränssnittet byggdes i *Pages/Index.cshtml*, medan tillhörande *Index.cshtml.cs* ansvarar för att hämta och förbereda den information som ska visas. Genom att använda de befintliga tjänsterna för kurser, uppgifter och prioritering kunde information från olika delar av applikationen samlas på en och samma sida.



Figur 7. StudyFlows personliga dashboard med prioriterade uppgifter, kommande deadlines, kursöversikt och navigation.

Dashboarden organiserades i flera sektioner. *Today's focus* visar den uppgift som prioriteringsalgoritmen har valt ut, medan *Coming up* presenterar kommande uppgifter med deadlines. Därutöver visas antalet slutförda uppgifter i *All the small things* och användarens kurser i *My little study corner (se figur 7)*. Uppgifterna presenteras med återanvändbara partial views, vilket gör att samma komponenter kan användas på flera sidor.

För navigationen skapades en gemensam layout i *Pages/Shared/_Layout.cshtml*, som innehåller sidornas övergripande struktur och navigationslänkar. Användaren kan navigera mellan dashboarden, kursöversikten och uppgiftssidan. Från kurskorten går det även att öppna respektive kurs detaljsida, där kursens egna uppgifter och funktioner finns samlade.

Genom att kombinera en gemensam layout med återanvändbara komponenter kunde gränssnittet hållas konsekvent samtidigt som varje sida presenterar information anpassad efter sitt användningsområde.

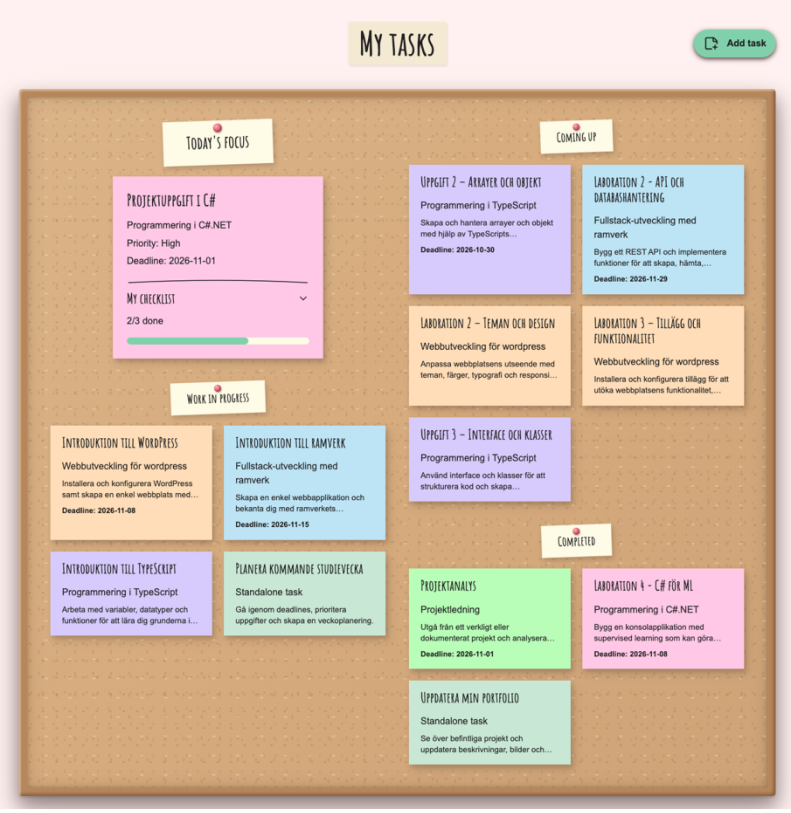
6.8 Gränssnitt och responsiv design

StudyFlows gränssnitt utvecklades iterativt med HTML, SCSS och JavaScript i kombination med Razor Pages. Utformningen utgick från de wireframes och mockups som tidigare tagits fram i Figma (se bilaga A), med målet att skapa en personlig studieplanerare med ett sammanhängande visuellt uttryck.

Dashboarden utformades med inspiration från en uppslagen anteckningsbok, medan uppgifter och kurser presenterades som kort med mjuka färger och detaljer som påminner om papper och anteckningar. För att skapa ett enhetligt gränssnitt användes återkommande färger, typografi, knappar och dialogrutor.

Stylingen organiserades med SCSS, där koden delades upp i separata filer för bland annat grundläggande typografi, layout, komponenter och enskilda sidor. Gemensamma färger och andra designvärden samlades i variabler för att underlätta justeringar och återanvändning. Denna uppdelning gjorde det möjligt att vidareutveckla gränssnittet utan att samla all styling i en enda fil.

För att anpassa applikationen till olika skärmstorlekar implementerades responsiv design med CSS media queries. Layout, navigering och innehåll anpassades för mindre skärmar, bland annat genom att justera placering och storlek på komponenter.



Figur 8. StudyFlows samlade uppgifter i desktopvy.



Figur 9. StudyFlows gränssnitt i mobilvy med responsivt anpassad layout.

7 Resultat

Projektet resulterade i StudyFlow, en fungerande webbaserad studieplanerare utvecklad med ASP.NET Core Razor Pages. Applikationen gör det möjligt för användare att registrera ett personligt konto, logga in och organisera sina studier genom att skapa och hantera kurser och studieuppgifter.

Användaren kan skapa uppgifter med beskrivning, deadline, prioritet och uppskattad arbetstid samt dela upp större uppgifter i mindre delmoment med hjälp av checklistor. Uppgifter kan kopplas till en kurs eller hanteras fristående, och deras status kan uppdateras under arbetets gång. Informationen lagras i en SQLite-databas och är knuten till respektive användarkonto.

En central del av resultatet är den regelbaserade prioriteringsalgoritmen, som automatiskt identifierar vilken uppgift användaren bör fokusera på. På den personliga dashboarden presenteras *Today's focus* tillsammans med kommande deadlines, slutförda uppgifter och användarens kurser. Prioriteringen används även på uppgiftssidan och de enskilda kurssidorna.

Det färdiga gränssnittet har ett sammanhängande visuellt uttryck och är responsivt anpassat för både större och mindre skärmar. Genom återanvändbara komponenter och dialogrutor kan användaren hantera stora delar av sitt studiearbete utan att behöva navigera mellan flera olika sidor.

Projektets huvudsakliga mål har därmed uppnåtts. StudyFlow innehåller de planerade funktionerna för personlig studieplanering, men applikationen har inte publicerats på någon extern webbserver. Kontobekräftelse hanteras lokalt eftersom ingen extern e-posttjänst har integrerats.

8 Slutsatser

Målet med projektet var att utveckla en personlig studieplanerare med funktioner för att organisera kurser, uppgifter och checklistor. Jag bedömer att projektets huvudsakliga mål har uppnåtts och att resultatet är en fungerande webbapplikation med de funktioner som planerades.

Projektet blev betydligt mer omfattande än jag hade föreställt mig från början. Framför allt tog arbetet med användarhantering, databaskopplingar och samspelet mellan gränssnitt och serverlogik mer tid än förväntat. Under utvecklingen växte även behovet av att strukturera om koden, vilket ledde till att flera större filer delades upp i mindre komponenter och tjänster. Detta gav mig en bättre förståelse för vikten av en tydlig kodstruktur och hur olika delar av en applikation påverkar varandra.

En av de största lärdomarna har varit att arbeta med C# i en webbaserad applikation och att använda ASP.NET Core Razor Pages. Eftersom jag tidigare arbetat med Angular var jag van vid en tydligare uppdelning mellan frontend och backend. Razor Pages upplevdes därför till en början som mindre intuitivt, framför allt eftersom gränssnitt och serverlogik hanteras inom samma projekt och tillhörande filer har liknande namn. Samtidigt har jag fått en större förståelse för fördelarna med att samla båda delarna i en och samma applikation, vilket förenklar utvecklingen på flera sätt.

Trots att projektet stundtals varit krävande har arbetet varit både roligt och lärorikt. Jag tycker särskilt om att utveckla applikationer med grafiska gränssnitt, där funktionaliteten får en visuell form och resultatet blir mer konkret än i en konsolapplikation.

Min ursprungliga ambition var även att publicera StudyFlow så att applikationen kunde användas via internet. Under projektets gång blev det dock tydligt att publiceringen skulle kräva ytterligare arbete, och jag valde därför att prioritera applikationens funktioner, användarupplevelse och stabilitet inom den tillgängliga tidsramen.

Vid en fortsatt utveckling skulle nästa steg vara att publicera applikationen och integrera en extern e-posttjänst för kontobekräftelse. Det skulle göra StudyFlow mer användbar utanför den lokala utvecklingsmiljön.

Sammanfattningsvis har projektet gett mig en större förståelse för hur en komplett webbapplikation byggs med C# och .NET, från datamodeller och serverlogik till ett fungerande användargränssnitt. Trots att arbetet blev mer omfattande än planerat är jag nöjd med slutresultatet och de kunskaper jag har fått under utvecklingen.

9 Källförteckning

[1] **Microsoft | Learn** - ***C# Introduktion:*** https://learn.microsoft.com/sv-se/dotnet/csharp/language-reference/language-specification/introduction
Senast uppdaterad: 2026-09-22. Hämtad: 2026-10-28.

[2] **Microsoft | Learn – *Introduktion till .NET:*** https://learn.microsoft.com/sv-se/dotnet/core/introduction
Senast uppdaterad: 2026-03-14. Hämtad: 2026-10-28.

[3] **Microsoft | Learn – *Översikt över ASP.NET Core:*** https://learn.microsoft.com/sv-se/aspnet/core/overview?view=aspnetcore-10.0
Senast uppdaterad: 2026-10-10. Hämtad 2026-10-28.

[4] **Microsoft | Learn – *Razor Pages-arkitektur och begrepp i ASP.NET Core:*** https://learn.microsoft.com/sv-se/aspnet/core/razor-pages/?view=aspnetcore-10.0&tabs=visual-studio
Senast uppdaterad: 2026-10-10. Hämtad 2026-10-28.

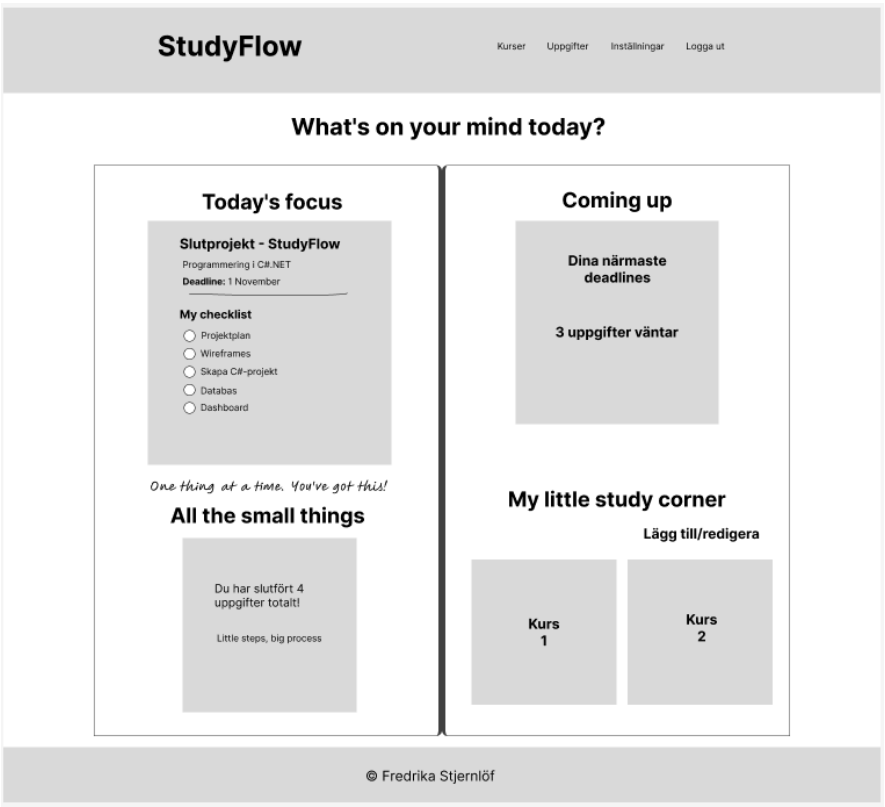
[5] **Microsoft | Learn – *Entity Framework Core:*** https://learn.microsoft.com/sv-se/ef/core/
Senast uppdaterad: 2025-11-02. Hämtad 2026-10-28.

[6] **Microsoft | Learn** - ***SQLite EF Core-databasprovider*:** https://learn.microsoft.com/sv-se/ef/core/providers/sqlite/?tabs=dotnet-core-cli
Senast uppdaterad: 2025-07-25. Hämtad 2026-10-28.

[7] **Microsoft | Learn** - ***Introduktion till Identity på ASP.NET Core:*** https://learn.microsoft.com/sv-se/aspnet/core/security/authentication/identity?view=aspnetcore-10.0&tabs=visual-studio
Senast uppdaterad: 2026-10-08. Hämtad 2026-10-28.

[8] **Service Innovation Lab –*Scoring Model: Guide, Practical Ecample & Method Comparison:*** https://www.si-labs.com/en/articles/scoring-model/
Publicerad: 2026-04-10. Hämtad 2026-10-28.

Bilaga A



Figur A. 1: Exempel på wireframe av StudyFlows startsida.



Figur A. 2: Exempel på Mockup av StudyFlows startsida, desktop-vy.



Figur A. 3: Exempel på mockup av StudyFlows startsida, mobil-vy.